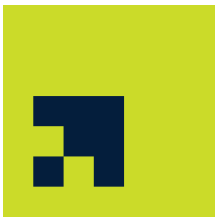


Gisma
University
of Applied
Sciences

Training GANs & VAEs with MLX for iOS Deployment

Professor: **Prof. Dr. Sami Al-Salamin**

M608 Business Project in Computer Science

Author

GH1031360 Sharan Deepak Thakur

sharan.thakur@gisma-student.com

Date: December 2025

GitHub Repository: https://github.com/c2p-cmd/all_about_gans

YouTube Video: https://www.youtube.com/watch?v=-rIP5dT_nKY

Abstract

The rapid evolution of Generative AI has traditionally relied on massive cloud-based GPU clusters, raising concerns regarding data privacy, latency, and operational costs. This report details the development of "ImageGen with MLX", a mobile application that challenges this cloud-dependency paradigm.

Leveraging Apple's new MLX framework (Hannun et al., 2023), this project demonstrates the feasibility of training Deep Learning models focused specifically on Generative Adversarial Networks (GANs) (Goodfellow et al., 2014) and Variational Autoencoders (VAEs) (Kingma and Welling, 2013) natively on macOS without NVIDIA hardware. The trained models were subsequently deployed to a custom iOS application for offline, on-device inference.

The solution was tested across four datasets: MNIST (LeCun et al., 1998), Fashion MNIST (Xiao et al., 2017), CIFAR-10 (Krizhevsky, 2009), and QuickDraw (Google Creative Lab, 2018). The resulting artifact serves as a proof-of-concept for organizations aiming to implement private, cost-effective, and edge-native AI solutions, bridging the gap between high-performance computing and consumer mobile hardware.

Contents

1 Introduction **3**
1.1 Topic and Context . . . 3
1.2 Problem Statement . . . 3
1.3 Project Objectives . . . 3
1.4 Research Questions . . . 3

2 Background and State of the Art **4**
2.1 The Evolution of Edge AI . . . 4
2.2 Justification for MLX . . . 4
2.3 Generative Architectures . . . 4

3 Methodology and System Design **5**
3.1 Dataset Characterization . . . 5
3.2 Platform Definition and Technology Stack . . . 5
3.3 Solution Implementation Stages . . . 6
3.3.1 Stage 1: The Training Pipeline in Python . . . 6
3.3.2 Stage 2: Model Architecture (DCGAN & VAE) . . . 8
3.3.3 Stage 3: The "Bridge" and iOS Implementation . . . 9

4 Results and Critical Evaluation **9**
4.1 RQ1: Training Efficiency on Apple Silicon . . . 9
4.2 RQ2: Qualitative Analysis (GAN vs. VAE) . . . 9
4.3 RQ3: Mobile Deployment and Usability . . . 11
4.4 Critical Evaluation . . . 12

5 Conclusion **13**
5.1 Future Work . . . 13

6 Project Deliverables & Source Code **14**

1 Introduction

1.1 Topic and Context

By 2025, Generative AI has become a cornerstone of modern software. However, the dominant training pipeline remains tethered to heavy cloud infrastructure (AWS, Azure) utilizing NVIDIA GPUs. For a trusted programmer and computing specialist, this creates a "Cloud Dependency Trap," where data must leave the organization's perimeter, incurring bandwidth costs and privacy risks. The emergence of Apple Silicon (M-series chips) offers a new paradigm: Unified Memory Architecture that allows the CPU and GPU to share resources efficiently.

1.2 Problem Statement

The core business problem addressed by this project is the **"Privacy-Performance Trade-off."** Organizations wish to deploy generative tools but hesitate due to the security risks of sending sensitive data to the cloud. While pre-trained models exist for mobile, the *training* phase is almost exclusively cloud-bound. There is a lack of documented pipelines that allow for the *entire* lifecycle going from training to deployment, to occur within a private, local ecosystem.

1.3 Project Objectives

The primary objective is to build a "Full-Stack Edge AI" system. The project aims to achieve the following:

- **Native Training:** Utilize the MLX framework to train generative models (GANs, VAEs) directly on macOS.
- **Algorithm Diversity:** Implement Deep Convolutional GANs (DCGAN) and VAEs to compare architectural performance.
- **Mobile Deployment:** Develop a native iOS application ("ImageGen") using SwiftUI to serve these models for real-time inference.
- **Multi-Domain Validation:** Test the models across diverse datasets (B&W Digits, B&W Fashion, Color Objects, B&W Sketches).

1.4 Research Questions

To guide the evaluation, the following research questions were defined:

- **RQ1 (Feasibility):** Is Apple's MLX framework a viable alternative to PyTorch/Tensorflow for training deep generative models on consumer hardware?
- **RQ2 (Quality Analysis):** How does the visual fidelity of VAEs compare to DC-GANs when constrained by mobile-compatible architectures?
- **RQ3 (Deployment):** Can complex generative models run with low latency on an iPhone without internet connectivity?

2 Background and State of the Art

2.1 The Evolution of Edge AI

Historically, mobile devices were only "viewers" for AI processed in data centers. State-of-the-Art (SOTA) solutions now attempt "Edge Inference," where the model runs on the phone. However, training remains the bottleneck. This project pushes the boundary by moving the *training* workload to local Apple Silicon machines, enabling a fully offline research-to-product cycle.

2.2 Justification for MLX

For this project, **Apple MLX** was selected over TensorFlow or PyTorch.

1. **Unified Memory:** MLX allows arrays to live in a shared memory space, therefore preventing the data copying between CPU and GPU that slows down traditional frameworks.
2. **Familiarity:** It offers a Python API similar to NumPy and PyTorch, ensuring code readability while unlocking hardware acceleration.
3. **Privacy:** It ensures that, all computation happens on-device, adhering to data isolation.

2.3 Generative Architectures

- Goodfellow et al. (2014) introduced **GANs (or Generative Adversarial Networks)**.These consist of two networks (Generator vs. Discriminator) fighting against each other. They typically produce sharper images but are harder to train correctly. This arises because one must train and tune two different neural networks of different architectures to ensure stability in learning as well as ensuring good performance.
- Kingma and Welling (2013) introduced **VAEs (Variational Autoencoders)**. These learn the probability distribution of the data. While more stable to train, they are

known to produce “blurrier“ results. Since they learn distribution of probabilities and hence concepts like “shapes“ are difficult to compress. They also consist of two neural networks but are not adversaries but co-operate with each other; the encoder encodes the images into a normal distribution (Wikipedia contributors, 2025) then this normal distribution with the mean and standard deviation goes to the decoder network which reconstructs the image.

- During generation the generator/decoder simply generates an image from a random noise normal distribution.

3 Methodology and System Design

3.1 Dataset Characterization

To ensure the models were robust, four distinct datasets were utilized, representing different complexity levels:

- **MNIST:** Handwritten digits (Grayscale, Low complexity).
- **Fashion MNIST:** Clothing items (Grayscale, Medium complexity).
- **CIFAR-10:** Real-world objects like cars and animals (Color, High complexity).
- **QuickDraw:** A subset of 8 classes (Cat, Basketball, Bird, Sword, Ice Cream, Mug, Fish, Stop Sign) from Google's sketch dataset.

3.2 Platform Definition and Technology Stack

- **Training Hardware:** MacBook Pro 14 with M3 Pro Chip and 18GB RAM running macOS 26.
- **Training Framework:** Python 3.11 + Apple MLX.
- **Application Layer:** Swift + SwiftUI for the iOS interface running on iPhone 16 with iOS 26.
- **Inference Engine:** MLX Swift.

3.3 Solution Implementation Stages

3.3.1 Stage 1: The Training Pipeline in Python

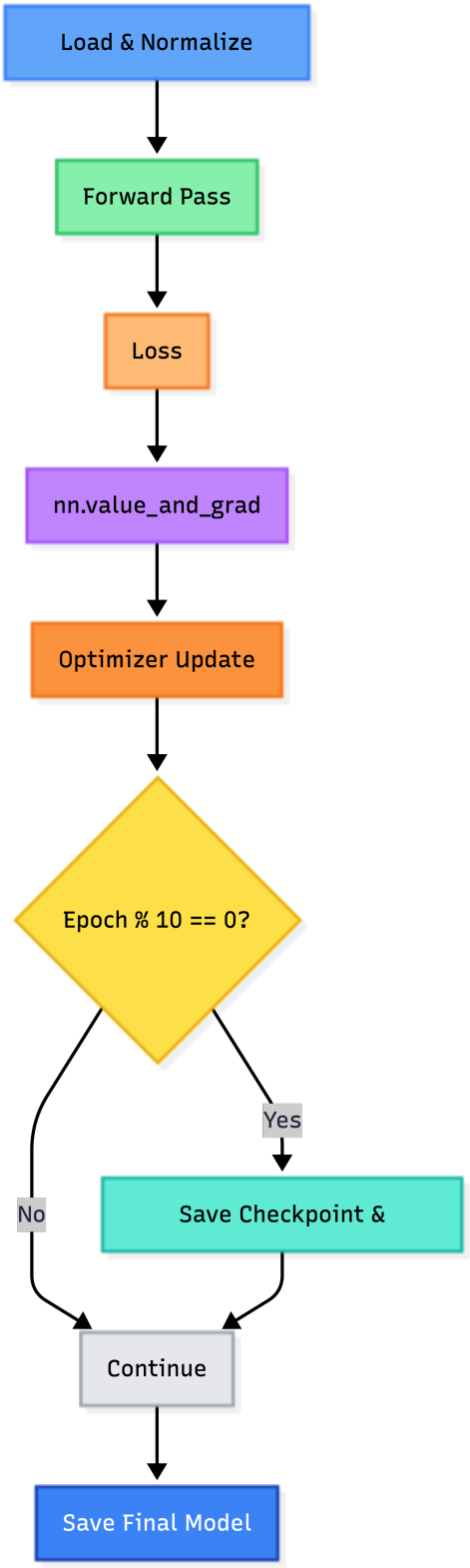


Figure 1: MLX Training Loop Implementation

The data is loaded and normalized based on model architecture. The MLX training loop computes gradients and the loss efficiently using the unified memory.

Addressing Training Instability: A common challenge in GAN training is the discriminator overpowering the generator, leading to vanishing gradients. To mitigate this, distinct strategies were employed:

- **Label Smoothing:** Instead of using hard labels (1.0 for real, 0.0 for fake), random smoothing was applied. Real images were labeled between $0.7 - 0.9$ and fake images between $0.1 - 0.2$. This prevents the discriminator from becoming too confident too quickly.
- **Learning Rate Tuning:** The discriminator's learning rate was lowered relative to the generator to maintain an equilibrium in the adversarial game.

Every 10th epoch, weights are saved alongside generated samples to monitor progress (Fig. 1).

3.3.2 Stage 2: Model Architecture (DCGAN & VAE)

DCGAN: Implemented with convolutional layers to capture spatial hierarchies in images.
VAE: Implemented with an encoder-decoder structure.

Table 1: Comparison of DCGAN and VAE Implementations for CIFAR-10

Aspect	**DCGAN (CIFAR-10)**	**VAE (CIFAR-10)**
Learning Objective	Adversarial training between Generator and Discriminator	Variational inference via reconstruction and regularization
Loss Function	Adversarial loss based on binary cross-entropy	Reconstruction loss + KL divergence
Data Normalization	$(-1, 1)$ to match `tanh` generator output	$(0, 1)$ to match `sigmoid` decoder output
Latent Dimension	100	200
Latent Sampling	Noise sampled from $\mathcal{N}(0, 1)$	Reparameterization: $z = \mu + \sigma \cdot \epsilon$
Encoder	None	CNN encoder with Conv, BatchNorm, and Pooling layers
Decoder / Generator	UpsamplingConv2D blocks with BatchNorm	UpsamplingConv2D blocks with BatchNorm
Output Activation	`tanh`	`sigmoid`
Model Outputs	Generated image	Reconstructed image, μ, and $\log \sigma^2$
Training Stability	Sensitive to hyperparameters and training imbalance	More stable due to explicit likelihood objective
Inference Sampling	Sample noise $\rightarrow$ Generator	Sample $z \sim \mathcal{N}(0, I) \rightarrow$ Decoder
Gradient Computation (MLX)	`nn.value_and_grad` with manual optimizer step	`nn.value_and_grad` with manual optimizer step
Lazy Execution Control	Explicit `mx.eval(parameters)`	Explicit `mx.eval(parameters)`
Checkpoint Artifacts	Generator weights and generated samples	Encoder and Decoder weights with reconstructions

3.3.3 Stage 3: The "Bridge" and iOS Implementation

The transition from the Python development environment to the production iOS environment required a custom engineering pipeline, as there is no automatic export tool for MLX-to-Swift.

1. Architecture Porting & Type Safety: The model architecture defined in Python had to be manually re-written in Swift. Unlike Python's dynamic typing, Swift enforces strict type checking. This required ensuring that every convolutional layer, batch normalization, and activation function in Swift matched the Python definition exactly in terms of kernel size, stride, and padding.

2. Weight Serialization with Safetensors: While `.npz` is supported, the complex module tree called for a more standard format. Weights were serialized using `safetensors` (Huggingface, 2022) in Python. This format provides a standard safe, zero-copy variable sharing, which is critical for preserving the integrity of nested tensor dictionaries.

3. Recursive Weight Loading: A significant challenge arose from version discrepancies between `mlx-python` and `mlx-swift`. To resolve this, a custom recursive loading algorithm was implemented in Swift. This method traverses the module tree, matching layer names and parameter shapes manually to ensure that the pre-trained weights were applied into the correct neural network layers in the Swift model.

4. User Interface: The final iOS app, "ImageGen with MLX", features a SwiftUI interface allowing users to select the model and batch size (2-64) for real-time inference.

4 Results and Critical Evaluation

4.1 RQ1: Training Efficiency on Apple Silicon

The project successfully demonstrated that MLX is highly capable. Training on the MNIST dataset took $16 mins$ for $20 epochs$ at a batch size of 64, and even the more complex CIFAR-10 dataset took $32 mins$ for $50 epochs$ at a batch size of 64 lastly, Quick Draw took $20 mins$ for $150 epochs$ at batch size of 256. The transition from Python training to Swift deployment although came with challenges, but are not impossible; validating the ecosystem's readiness for business applications.

4.2 RQ2: Qualitative Analysis (GAN vs. VAE)

A visual inspection of the generated images confirms theoretical distinctions:

- **CIFAR-10 Results:** The **DCGAN** successfully generated recognizable shapes (e.g., buildings, animals). In contrast, the **VAE** struggled with the color data,

producing "blurrier" images with undefined edges.

- **Fashion MNIST:** The VAE performed better here than on CIFAR, generating recognizable purses and heels, though still lacking the sharpness of the GAN.
- **QuickDraw:** The models successfully learned abstract concepts, generating clear sketches of "swords" and "mugs."

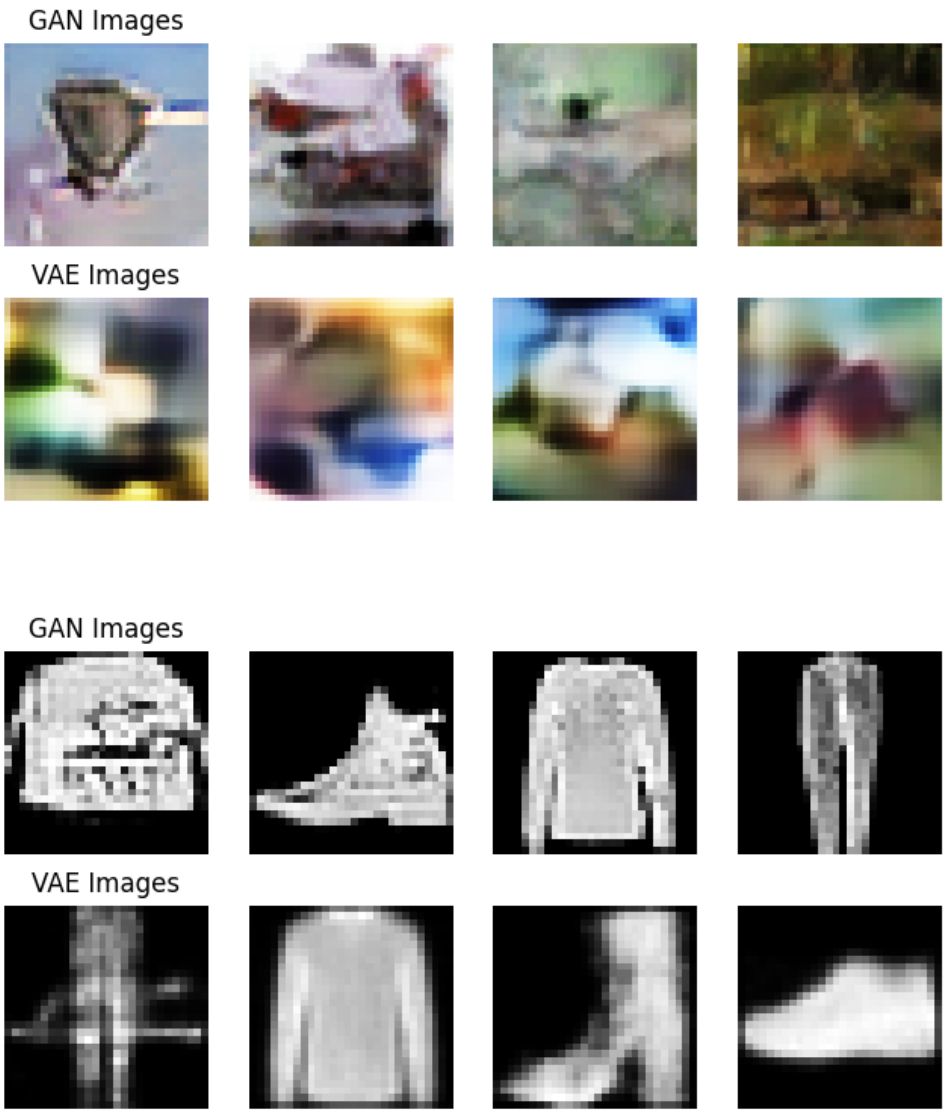


Figure 2: Comparison of VAE (Blurry) vs GAN (Sharp) outputs on CIFAR-10 and Fashion MNIST

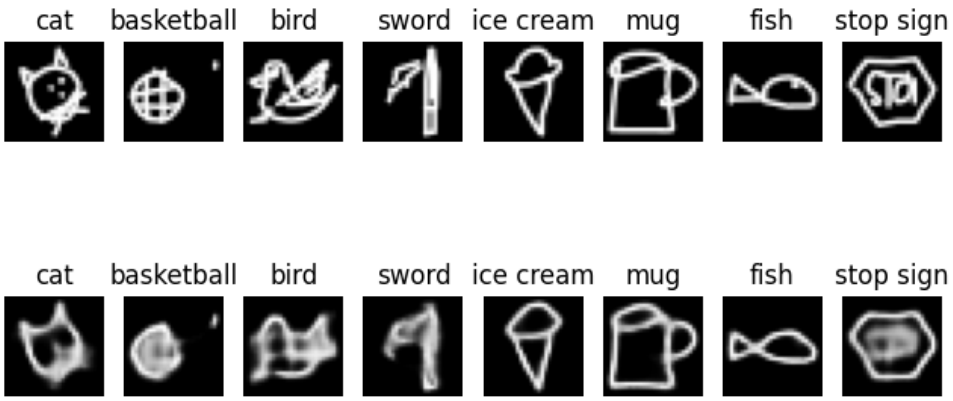


Figure 3: Quick Draw VAE reconstructed images for each class

4.3 RQ3: Mobile Deployment and Usability

The "ImageGen" iOS app proved that inference is highly efficient on the iPhone 16.

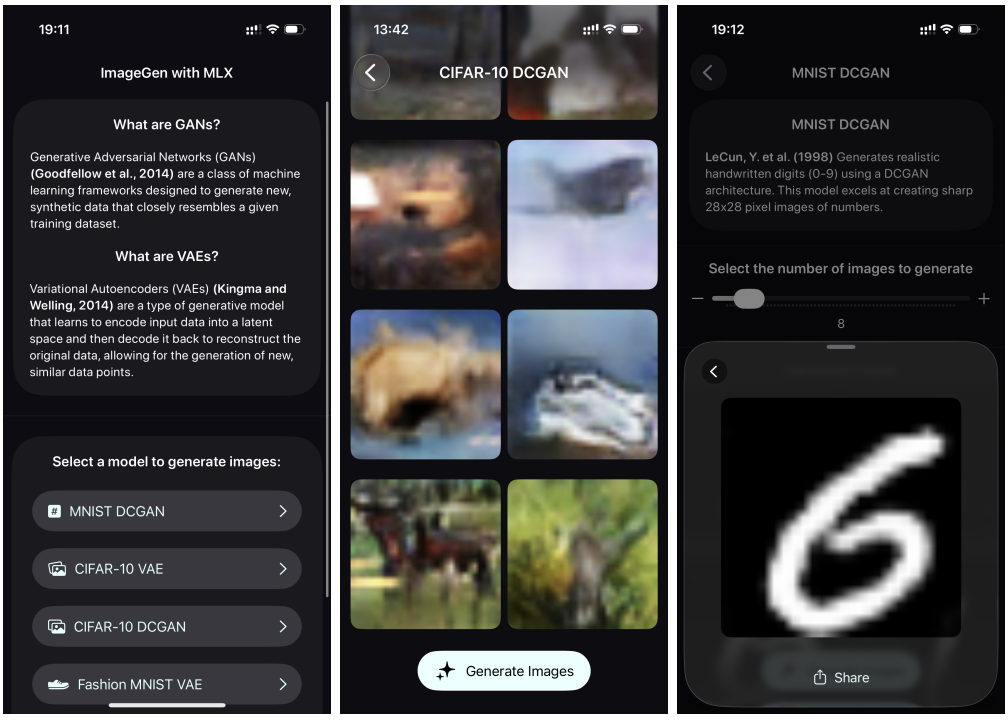
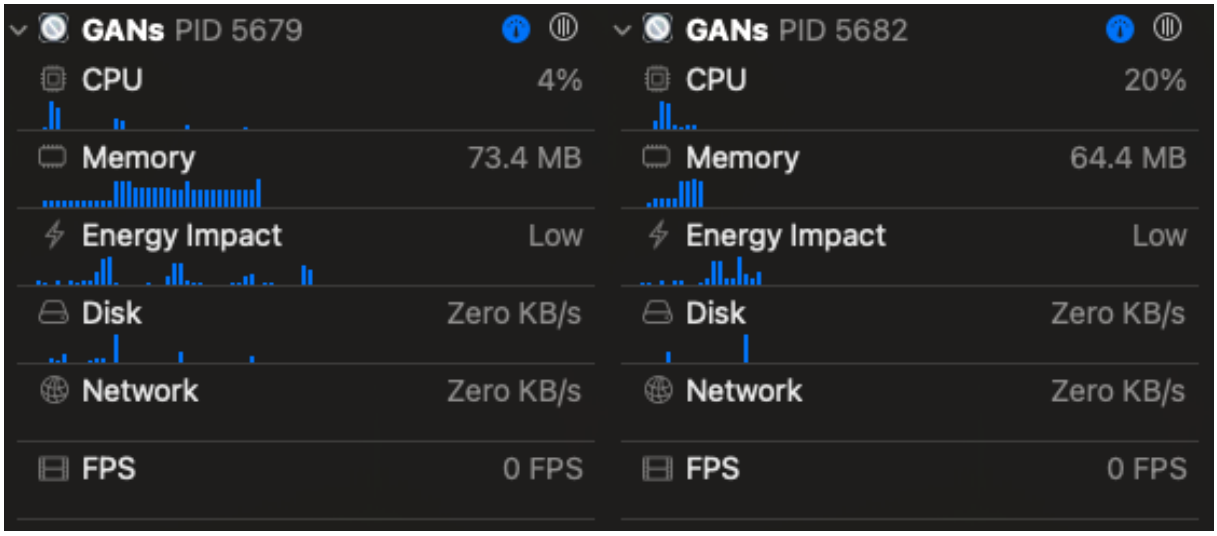


Figure 4: Mobile App Screenshots running on iPhone 16

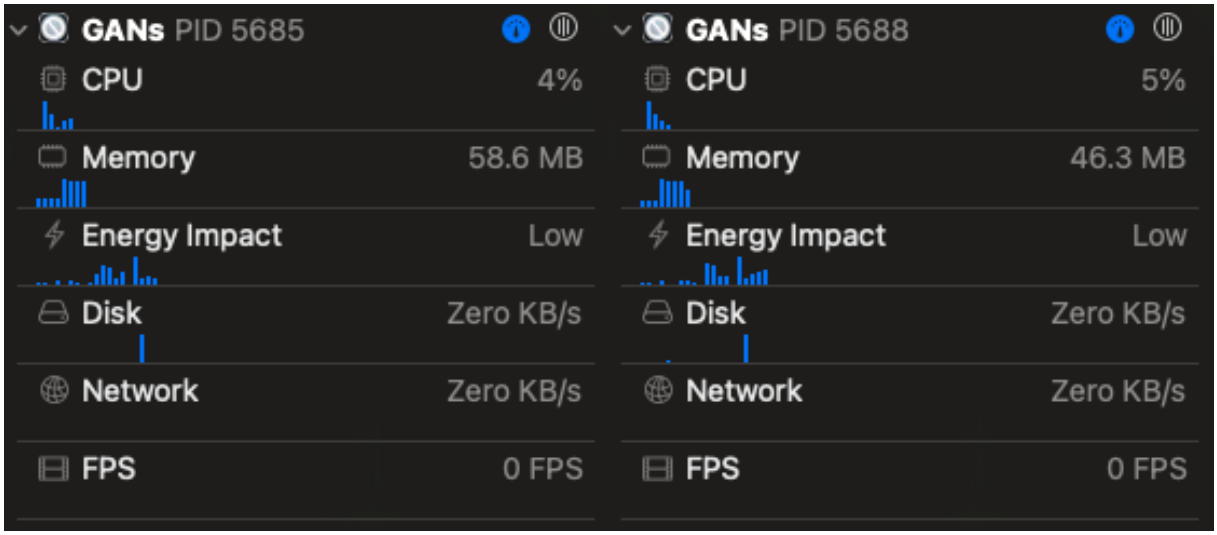
- **Latency:** Generating a batch of 12 images happens in milliseconds, perceptibly instant to the user.
- **Memory Efficiency:** Profiling the application revealed a highly optimized memory

footprint. The more complex Color models (CIFAR-10) peaked between **64-73 MB** (Fig. 5a & 5b) of unified memory usage, while the Grayscale models stabilized around **46-58 MB** (Fig. 5c & 5d). This low footprint validates that on-device generative AI is viable even on older hardware with limited RAM (Fig. 5).

- **User Experience:** The app allows users to "Share" (Fig 4) images immediately, proving that the generated data is standard image media usable by other apps.



(a) CIFAR 10 VAE Memory (b) CIFAR 10 DCGAN Memory



(c) MNIST VAE Memory (d) MNIST DCGAN Memory

Figure 5: Memory consumption across models

4.4 Critical Evaluation

While the prototype is functional, the following limitations were observed:

1. **Detail Fidelity:** VAEs on mobile architectures struggle with high-frequency details

(sharp edges), resulting in a "washed out" look for complex objects.

2. **Resolution:** The current app generates low-resolution images (28x28 or 32x32). Upscaling algorithms would be required for commercial graphic design use.

5 Conclusion

This project successfully achieved its primary objective: to design and implement a "Full-Stack Edge AI" solution that challenges the industry's reliance on cloud infrastructure. By architecting a complete pipeline which serves as a robust proof-of-concept for the "Privacy-Performance Trade-off".

The results validate that modern consumer hardware has crossed a critical threshold. The successful training of complex GANs and VAEs on a MacBook Pro, followed by real-time inference on an iPhone with just $\approx 70MB$ of memory overhead, demonstrates that organizations can now decouple their AI strategy from expensive, centralized GPU clusters.

Specifically, the project delivers three key business values:

- **Autonomy:** By removing the dependency on external cloud providers (AWS/Azure), the organization retains full control over its intellectual property and operational costs.
- **Privacy:** This architecture ensures that sensitive training data never leaves the local machine, and user prompts never leave the mobile device. This opens new market opportunities in highly regulated sectors like healthcare or finance where cloud AI is often non-compliant (Murdoch, 2024).
- **Resilience:** The application's ability to function fully offline ensures business continuity in remote or disconnected environments, a capability that cloud-tethered solutions cannot match.

In summary, "ImageGen with MLX" is not merely a technical demo; it is a blueprint for a sustainable, private, and cost-effective AI ecosystem.

5.1 Future Work

To evolve this prototype into a production-ready enterprise tool, the following enhancements are proposed:

- **Super-Resolution Upscaling:** Currently, the model outputs are limited to 32×32 pixels. Integrating a Super-Resolution GAN (SRGAN) on the device could upscale

these outputs to HD quality in real-time, making them suitable for commercial graphic design.

- **Federated Learning:** To further leverage the edge architecture, a Federated Learning protocol could be implemented. This would allow the iOS application to fine-tune the model on user data locally and share only the weight updates (encrypted) back to the central server, enabling the model to "learn" from thousands of devices without ever seeing a single private image.
- **Text-to-Image Synthesis:** Implementing a Diffusion model (e.g., Stable Diffusion) using the same MLX pipeline would allow for precise control over generation via text prompts, significantly expanding the tool's utility for creative professionals.

6 Project Deliverables & Source Code

The complete source code, including the MLX training scripts and the Xcode iOS project, is available here:

GitHub Repository:
https://github.com/c2p-cmd/all_about_gans/

YouTube Video:
https://www.youtube.com/watch?v=-rIP5dT_nKY

References

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). Generative adversarial nets. In *Advances in Neural Information Processing Systems*, volume 27.

Google Creative Lab (2018). The quick, draw! dataset. https://github.com/googlecreativelab/quickdraw-dataset. Accessed: 2025-12-14.

Hannun, A., Garg, J., Lavril, T., et al. (2023). Mlx: An array framework for apple silicon. https://github.com/ml-explore/mlx. Accessed: 2025-12-14.

Huggingface (2022). *safetensors: Safetensors File Format*.

Kingma, D. P. and Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. *Technical Report, University of Toronto*, pages 32–33.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324.

Murdoch, B. (2024). Ai adoption lags in regulated sectors amid compliance demands. *IT Brief*. Accessed: 2025-12-14.

Wikipedia contributors (2025). Normal distribution.

Xiao, H., Rasul, K., and Vollgraf, R. (2017). Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*.